

JavaScript Inheritance Types

Assignment 1 - Full-Stack Development Lab

Single

Multilevel

Hierarchical

Multiple

Hybrid

Anil Neerukonda Institute of Technology & Sciences (ANITS)

Programming Language: JavaScript (ES6+ Classes)

1. Single Inheritance

In Single Inheritance, a child class inherits from exactly ONE parent class. The child gets all properties and methods of the parent, and can also override them or add its own. Here, Dog extends Animal.

CODE:

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }
  speak() { return `${this.name} says "${this.sound}"`; }
  breathe() { return `${this.name} breathes air.`; }
}

// Dog inherits from Animal (Single Inheritance)
class Dog extends Animal {
  constructor(name, breed) {
    super(name, "Woof"); // calls parent constructor
    this.breed = breed;
  }
  fetch() { return `${this.name} (${this.breed}) fetches the ball!`; }
  speak() { return `${this.name} barks loudly: "${this.sound}!"`; } // override
}

const animal = new Animal("Cat", "Meow");
const dog = new Dog("Rex", "German Shepherd");
console.log(animal.speak());
console.log(dog.speak()); // overridden
console.log(dog.breathe()); // inherited from Animal
console.log(dog.fetch()); // own method
```

OUTPUT:

```
=== SINGLE INHERITANCE ===

Parent class (Animal):
  Cat says "Meow"
  Cat breathes air.

Child class (Dog extends Animal):
  Rex barks loudly: "Woof!"
  Rex breathes air.
  Rex (German Shepherd) fetches the ball!

instanceof Animal? true
instanceof Dog?    true
```

2. Multilevel Inheritance

In Multilevel Inheritance, classes form a chain: A -> B -> C.

Each level extends the one above. The deepest child inherits from ALL ancestors. Here: Person -> Employee -> Manager.

CODE:

```
class Person {
  constructor(name, age) { this.name = name; this.age = age; }
  introduce() { return `Hi, I am ${this.name}, ${this.age} years old.`; }
  eat()       { return `${this.name} is eating.`; }
}

// Level 1: Employee extends Person
class Employee extends Person {
  constructor(name, age, company, salary) {
    super(name, age);
    this.company = company; this.salary = salary;
  }
  work()      { return `${this.name} works at ${this.company}.`; }
  getSalary() { return `Salary: Rs.${this.salary.toLocaleString()}`; }
}

// Level 2: Manager extends Employee (multilevel chain)
class Manager extends Employee {
  constructor(name, age, company, salary, teamSize) {
    super(name, age, company, salary);
    this.teamSize = teamSize;
  }
  manage()    { return `${this.name} manages ${this.teamSize} people.`; }
  introduce() { return `Hi, I am ${this.name}, Manager at ${this.company}.`; }
}

const manager = new Manager("Carol", 40, "Infosys", 2000000, 15);
console.log(manager.introduce()); // overridden
console.log(manager.eat());       // from Person (2 levels up)
console.log(manager.work());      // from Employee
console.log(manager.manage());    // own method
```

OUTPUT:

```
=== MULTILEVEL INHERITANCE ===
Chain: Person -> Employee -> Manager

Level 0 - Person:
  Hi, I am Alice, 25 years old.
  Alice is eating.

Level 1 - Employee (extends Person):
  Hi, I am Bob, 30 years old.
  Bob works at TCS.
  Bob's salary: Rs.8,00,000
```

Level 2 - Manager (extends Employee):

Hi, I am Carol, Manager at Infosys.

Carol is eating.

Carol works at Infosys.

Carol's salary: Rs.20,00,000

Carol manages a team of 15 people at Infosys.


```
manager instanceof Person? true
```

```
manager instanceof Employee? true
```

```
manager instanceof Manager? true
```

3. Hierarchical Inheritance

In Hierarchical Inheritance, ONE parent class is inherited by MULTIPLE child classes independently. Each child shares parent behaviour but can add its own. Here: Vehicle -> Car, Bike, Truck.

CODE:

```
class Vehicle {
  constructor(brand, speed) { this.brand = brand; this.speed = speed; }
  move() { return `${this.brand} is moving at ${this.speed} km/h.`; }
  fuelType() { return `${this.brand} runs on fuel.`; }
}

// Child 1
class Car extends Vehicle {
  constructor(brand, speed, doors) { super(brand, speed); this.doors = doors; }
  honk() { return `${this.brand} car honks: Beep Beep!`; }
}

// Child 2
class Bike extends Vehicle {
  constructor(brand, speed, type) { super(brand, speed); this.type = type; }
  wheelie() { return `${this.brand} ${this.type} bike does a wheelie!`; }
}

// Child 3
class Truck extends Vehicle {
  constructor(brand, speed, payload) { super(brand, speed); this.payload = payload; }
  loadCargo() { return `${this.brand} truck loads ${this.payload} tons.`; }
}

const car = new Car("Toyota", 180, 4);
const bike = new Bike("Royal Enfield", 140, "Cruiser");
console.log(car.move()); // inherited
console.log(car.honk()); // own
console.log(bike.move()); // inherited
console.log(bike.wheelie()); // own
```

OUTPUT:

```
=== HIERARCHICAL INHERITANCE ===
Parent: Vehicle | Children: Car, Bike, Truck

--- Car (extends Vehicle) ---
Car: Toyota, 4 doors, 180 km/h
Toyota is moving at 180 km/h.
Toyota runs on fuel.
Toyota car honks: Beep Beep!

--- Bike (extends Vehicle) ---
Bike: Royal Enfield (Cruiser), 140 km/h
Royal Enfield is moving at 140 km/h.
Royal Enfield Cruiser bike does a wheelie!
```


--- Truck (extends Vehicle) ---

Tata is moving at 100 km/h.

Tata truck loads 20 tons of cargo.


```
car instanceof Vehicle? true
```

```
bike instanceof Vehicle? true
```

```
car instanceof Bike? false
```

4. Multiple Inheritance (Mixin Pattern)

JavaScript does NOT support native multiple inheritance.

The Mixin pattern is the standard JS approach: define behaviours as mixin functions and compose them onto a base class.

CODE:

```
// Mixin 1 - adds fly behaviour to any class
const Flyable = (Base) => class extends Base {
  fly() { return `${this.name} soars through the sky!`; }
};

// Mixin 2 - adds swim behaviour to any class
const Swimmable = (Base) => class extends Base {
  swim() { return `${this.name} dives in and swims!`; }
};

// Mixin 3 - adds run behaviour to any class
const Runnable = (Base) => class extends Base {
  run() { return `${this.name} runs at full speed!`; }
};

class Animal {
  constructor(name, species) { this.name = name; this.species = species; }
  describe() { return `${this.name} is a ${this.species}.`; }
}

// Duck inherits from ALL THREE mixins + Animal
class Duck extends Runnable(Swimmable(Flyable(Animal))) {
  constructor(n) { super(n, "Duck"); }
  quack() { return `${this.name} quacks: Quack Quack!`; }
}

const duck = new Duck("Donald");
console.log(duck.fly()); // from Flyable mixin
console.log(duck.swim()); // from Swimmable mixin
console.log(duck.run()); // from Runnable mixin
console.log(duck.quack()); // own method
```

OUTPUT:

```
=== MULTIPLE INHERITANCE (Mixin Pattern) ===
Mixins: Flyable, Swimmable, Runnable

--- Duck (Flyable + Swimmable + Runnable + Animal) ---
Donald is a Duck.
Donald spreads wings and soars through the sky!
Donald dives into the water and swims!
Donald runs at full speed!
Donald quacks: Quack Quack!

--- Eagle (Flyable + Runnable only) ---
Bald Eagle soars through the sky!
```

Bald Eagle hunts prey from above!

Can eagle swim? false

--- Fish (Swimmable only) ---

Nemo dives into the water and swims!

Can fish fly? false


```
duck instanceof Animal? true
```

```
'fly' in duck?      true
```

```
'swim' in duck?      true
```

5. Hybrid Inheritance

Hybrid Inheritance is a combination of two or more inheritance types.

Here we combine Hierarchical + Multilevel + Mixin:

Shape -> TwoDShape/ThreeDShape (hierarchical), then Circle/Rectangle extend TwoDShape (multilevel), and Sphere uses a Colorable mixin.

CODE:

```
// Mixin
const Colorable = (Base) => class extends Base {
  setColor(c) { this.color = c; return `Color set to ${c}.`; }
  getColor() { return `Color: ${this.color || 'none'}`; }
};

class Shape {
  constructor(name) { this.name = name; }
  area() { return 0; }
}

// Hierarchical split from Shape
class TwoDShape extends Shape {
  info() { return `${this.name} | 2D | Area=${this.area().toFixed(2)}`; }
}
class ThreeDShape extends Shape {
  volume() { return 0; }
  info() { return `${this.name} | 3D | Area=${this.area().toFixed(2)} | Vol=${this.volum
}

// Multilevel from TwoDShape
class Circle extends TwoDShape {
  constructor(r) { super("Circle"); this.radius = r; }
  area() { return Math.PI * this.radius ** 2; }
}
class Rectangle extends TwoDShape {
  constructor(w, h) { super("Rectangle"); this.width=w; this.height=h; }
  area() { return this.width * this.height; }
}

// Multilevel from ThreeDShape + Colorable mixin
class Sphere extends Colorable(ThreeDShape) {
  constructor(r, color) { super("Sphere"); this.radius=r; this.color=color; }
  area() { return 4 * Math.PI * this.radius ** 2; }
  volume() { return (4/3) * Math.PI * this.radius ** 3; }
}

const circle = new Circle(7);
const sphere = new Sphere(4, "Blue");
console.log(circle.info());
console.log(sphere.info());
console.log(sphere.getColor());
```

OUTPUT:

=== HYBRID INHERITANCE ===

--- Circle (Shape -> TwoDShape -> Circle) ---

Shape: Circle

Circle | 2D | Area=153.94 | Perimeter=43.98

```
instanceof Shape?    true
```

```
instanceof TwoDShape? true
```


--- Rectangle (Shape -> TwoDShape -> Rectangle) ---

Rectangle | 2D | Area=50.00 | Perimeter=30.00

```
instanceof TwoDShape? true
```


--- Sphere (Shape -> ThreeDShape -> Sphere + Colorable) ---

Color: Blue

Sphere color set to Gold.

```
instanceof ThreeDShape? true
```

```
'setColor' in sphere?    true
```